

Sistema de Drones

Proyecto 2



Informe De Desarrollo

Universidad San Carlos de
Guatemala Facultad de
ingeniería.

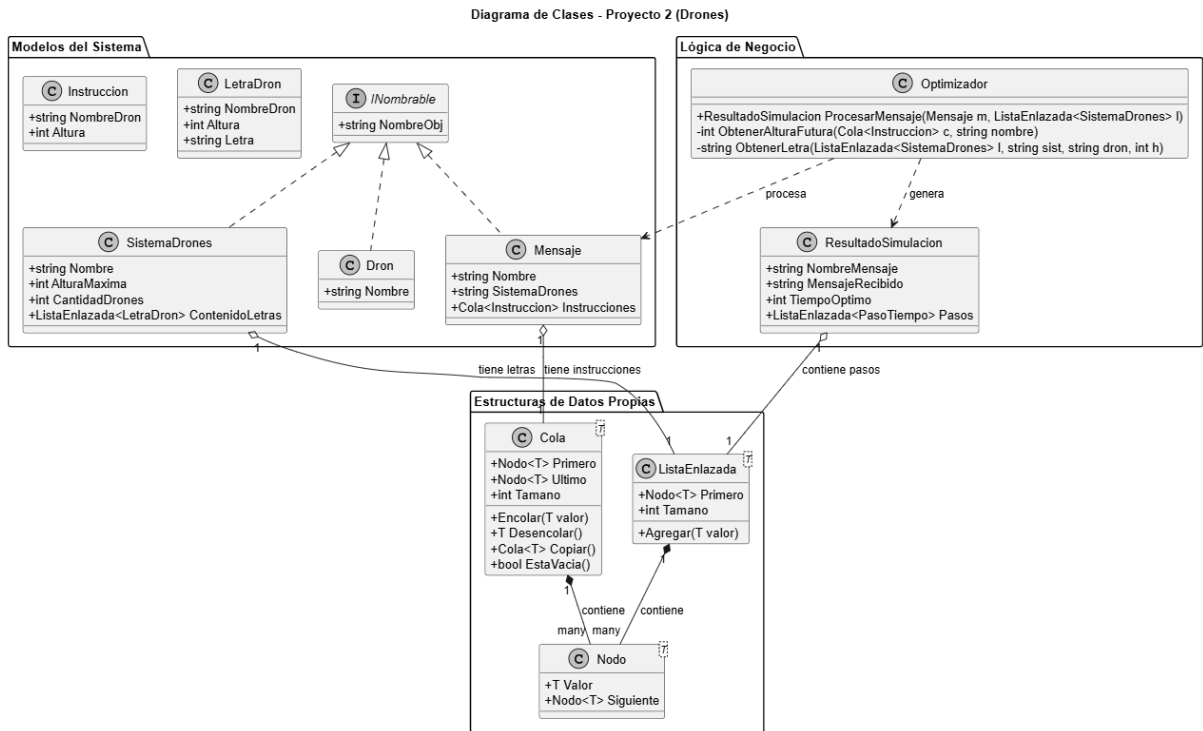
Introducción

El presente sistema surge como una respuesta tecnológica a la necesidad de optimizar el despliegue de enjambres de drones en misiones de comunicación visual. Mediante el uso de estructuras de datos lineales de memoria dinámica, se modela un entorno de simulación capaz de coordinar múltiples unidades de forma simultánea. El software no solo gestiona la logística de alturas y tiempos óptimos, sino que garantiza la integridad de los mensajes emitidos bajo estrictas reglas de negocio. Esta implementación demuestra la viabilidad de utilizar algoritmos predictivos para la resolución de problemas de ingeniería complejos en tiempo real. Finalmente, la integración de reportes gráficos y persistencia en XML asegura una herramienta de auditoría robusta y escalable para el usuario.

Objetivos

- Optimización de Estructuras de Datos Mediante TDA Propios: Implementar Tipos de Datos Abstractos (TDA) personalizados, específicamente Listas Enlazadas y Colas, mediante la gestión manual de memoria dinámica y punteros. El fin primordial es garantizar un control absoluto sobre el flujo de información y cumplir con las restricciones técnicas de no utilizar colecciones predefinidas del lenguaje C#, fortaleciendo así la capacidad de abstracción y lógica de programación a bajo nivel.
- Desarrollo de un Algoritmo de Simulación Predictiva: Diseñar un motor de simulación capaz de coordinar las trayectorias de un enjambre de drones de forma eficiente. Este objetivo busca reducir el tiempo óptimo de ejecución mediante una lógica de anticipación de movimientos, permitiendo que las unidades se desplacen hacia sus alturas objetivo de manera paralela y no secuencial, optimizando así los recursos operativos del sistema.
- Garantizar la Interoperabilidad y Validación de Datos: Establecer un estándar de comunicación robusto mediante la integración de archivos XML para la persistencia de datos y el uso de Graphviz para la generación de reportes gráficos. El objetivo es asegurar que la configuración de los sistemas de drones sea auditable y visualmente comprensible, permitiendo validar que las reglas de negocio y los estados de cada dron coincidan con la lógica programada en el backend.

Diagrama de Clases



Desarrollo del Proyecto

Sistema de Gestión de Drones

1. Implementación de Estructuras de Datos de Memoria Dinámica

Uno de los pilares fundamentales de este proyecto fue el rechazo total a las colecciones predefinidas del framework .NET. Siguiendo las restricciones de la cátedra, se diseñaron estructuras de datos propias basadas en nodos genéricos. La utilización de una Lista Enlazada Simple permitió gestionar los sistemas de drones y sus configuraciones de altura con una flexibilidad que las estructuras estáticas no poseen.

Por otro lado, la gestión de las instrucciones de los mensajes se realizó mediante una Cola (FIFO). Esta estructura resultó ser la más eficiente para el procesamiento de mensajes, ya que garantiza que las letras se emitan en el orden exacto en que fueron solicitadas, manteniendo la integridad del mensaje final "decodificado".

2. Algoritmo de Optimización y Simulación Segundo a Segundo

El núcleo del software reside en el Optimizador. A diferencia de una simulación lineal simple, este algoritmo implementa una lógica predictiva. Mientras un dron específico está ocupado emitiendo luz en una altura H en el segundo T , los demás drones del sistema no permanecen ociosos.

Mediante el método `ObtenerAlturaFutura`, el sistema escanea las instrucciones restantes en la cola. Si un dron detecta que su próxima participación requiere una altura distinta a la actual, inicia su ascenso o descenso de forma anticipada. Esta estrategia de "movimiento en paralelo" es la que permite minimizar el tiempo total de la misión, cumpliendo con el objetivo de eficiencia energética y operativa exigido por el Ministerio de la Defensa.

3. Procesamiento de Mensajes y Validación de Reglas de Negocio

El sistema garantiza que se cumplan estrictamente las restricciones físicas:

Unicidad de Emisión: Solo un dron puede emitir luz por segundo. Esto se controla mediante una bandera booleana (`luzEmitidaEnEsteSegundo`) que bloquea a otros drones de realizar la misma acción en el mismo instante de tiempo.

Gestión de Alturas: Los drones calculan su posición actual respecto a la objetivo, decidiendo dinámicamente entre las acciones de "Subir", "Bajar" o "Esperar".

Persistencia en XML: La arquitectura permite que, tras la simulación, todos los pasos almacenados en una lista de `ResultadoSimulacion` se serialicen en un archivo XML de salida, manteniendo la jerarquía de respuestas y mensajes solicitada.

4. Representación Gráfica con Graphviz

Para facilitar la auditoría del sistema, se integró la generación de archivos de texto en lenguaje DOT. Estos archivos permiten visualizar la configuración de los sistemas de drones como grafos dirigidos, donde cada nodo representa una altura y una letra asociada. Esta herramienta fue vital durante la etapa de pruebas para verificar que el cargue de datos desde el XML inicial fuera consistente con la estructura en memoria.

Ejecución y Compatibilidad de Software Usado

El desarrollo del sistema de optimización de drones se fundamenta en un ecosistema de herramientas de software de alto nivel, seleccionadas para garantizar un entorno de ejecución robusto, una interfaz de usuario intuitiva y una capacidad de procesamiento lógica eficiente. A continuación, se detallan los componentes técnicos que integran la solución:

1. Framework de Desarrollo: ASP.NET Core MVC

La aplicación fue construida utilizando el framework ASP.NET Core, bajo el patrón de arquitectura Model-View-Controller (MVC). Esta elección no es casual; permite una separación clara entre la lógica de las estructuras de datos (Modelos), la interfaz de usuario basada en Razor Pages (Vistas) y la gestión de peticiones que orquestan la simulación (Controladores). Al ser una tecnología multiplataforma, garantiza que el sistema pueda ejecutarse tanto en entornos Windows como en distribuciones Linux, cumpliendo con los estándares actuales de la industria.

2. Entorno de Ejecución: .NET 8.0 SDK

Para asegurar la compatibilidad con las últimas optimizaciones de rendimiento y seguridad, se utilizó el SDK de .NET 8.0. Este entorno proporciona el motor de ejecución (Runtime) necesario para procesar las operaciones de memoria dinámica y punteros requeridas por las listas enlazadas y colas personalizadas. La compatibilidad está garantizada en sistemas operativos modernos (Windows 10/11, macOS y Linux), siempre que cuenten con el runtime de .NET instalado.

3. Motor de Visualización: Graphviz (Lógica de Grafos)

La transparencia en el funcionamiento de los sistemas de drones se logra mediante la integración con Graphviz. El software utiliza el motor de diseño dot para transformar archivos de texto plano, generados algorítmicamente por la aplicación, en diagramas visuales en formato .png. Para una correcta ejecución, el sistema requiere que las variables de entorno del sistema operativo tengan acceso al ejecutable de Graphviz, permitiendo que el servidor web invoque los comandos necesarios para renderizar la estructura de los sistemas y los mensajes de forma dinámica.

4. Interfaz y Estilo: Bootstrap 5 y Razor

Para la capa de presentación, se utilizó Bootstrap 5, lo que permite una compatibilidad Responsive Design. Esto asegura que el panel de gestión de drones pueda ser visualizado y operado desde distintos dispositivos sin perder la integridad de la interfaz. El uso de componentes como Accordions y Modals facilita la navegación por las complejas estructuras de datos cargadas desde el XML sin saturar visualmente al usuario.

5. Herramientas de Persistencia y Datos: System.Xml

Aunque el proyecto prohíbe el uso de colecciones de C#, se apoya en la librería

estándar System.Xml para la interpretación de los archivos de entrada. Esta herramienta es compatible con el estándar XML 1.0, lo que permite que el software procese correctamente cualquier archivo generado por otras aplicaciones que sigan dicha norma, asegurando la interoperabilidad del sistema.

Versiones utilizadas:

- Lenguaje: C#
- Framework: .NET 8.0
- Editor de código: Visual Studio Code 1.80 o superior
- Extensión utilizada: C# Dev Kit
- Herramienta de visualización: Graphviz 10.x

Requisitos para ejecutar el proyecto:

Para ejecutar el proyecto en otro equipo es necesario contar con los siguientes elementos instalados:

- .NET SDK 8.0 o superior
- Visual Studio Code o cualquier IDE compatible con C#
- Graphviz instalado para generar los diagramas a partir del archivo .dot

Ejecución del proyecto:

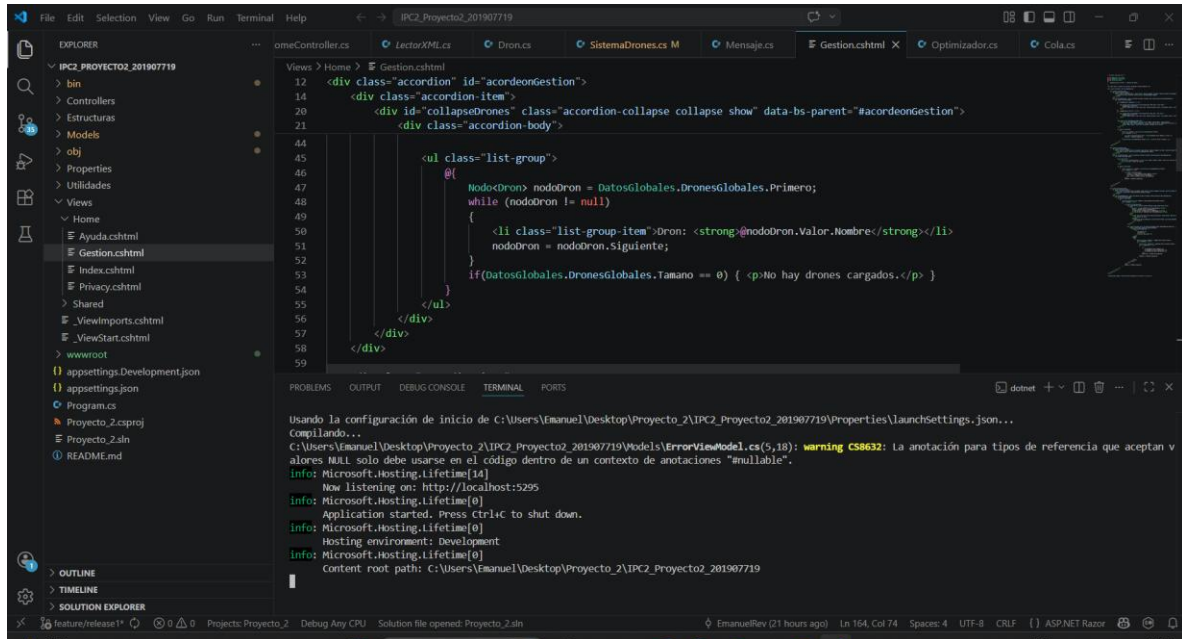
Una vez descargado el proyecto, se debe abrir la carpeta en el editor de código y ejecutar el programa desde la terminal con el siguiente comando:

dotnet run

El sistema mostrará un menú en consola desde el cual el usuario podrá cargar el archivo XML, ejecutar la simulación, generar los resultados y producir los diagramas correspondientes.

Funcionamiento de Programa

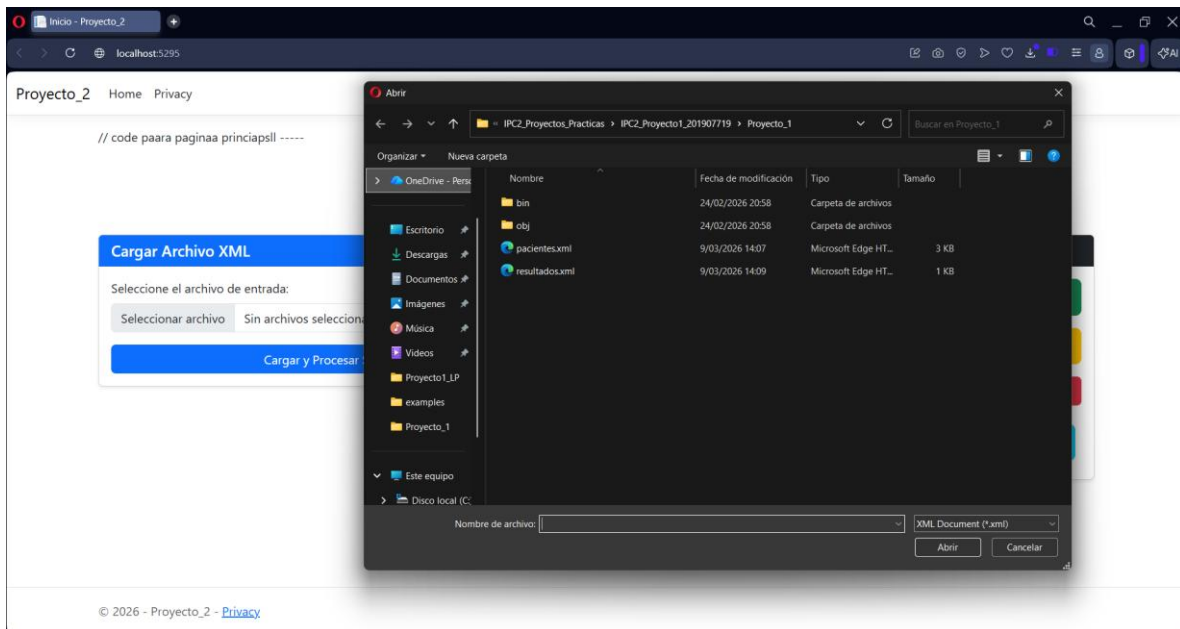
Se inicia el programa y se da click en el link que genera el programa.



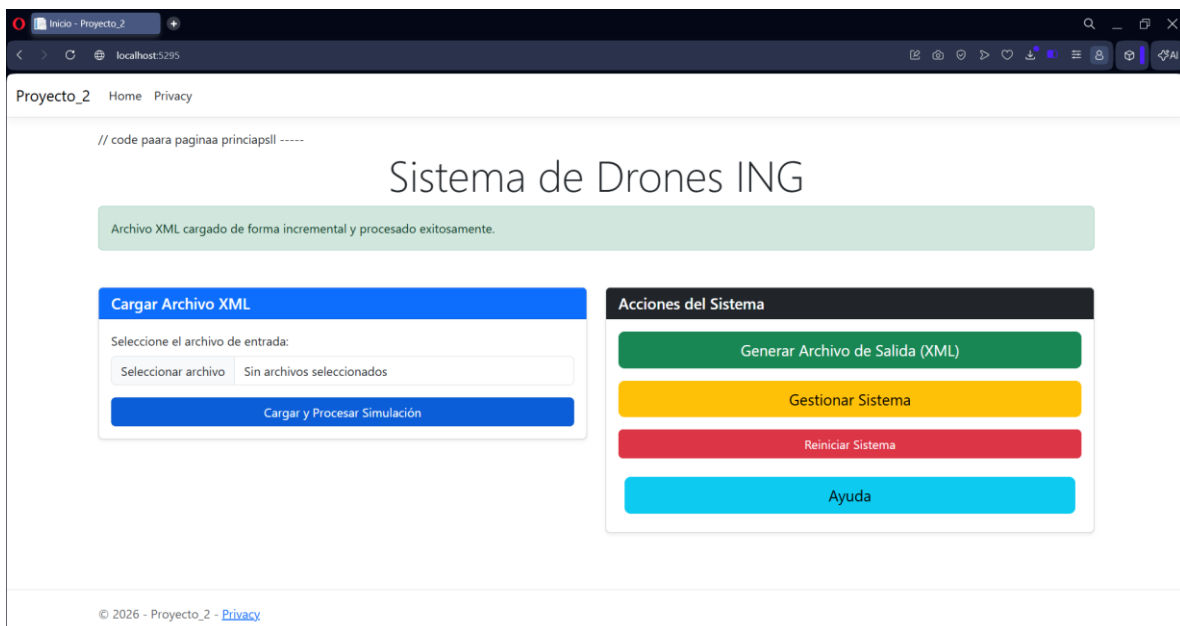
Se abre nuestro navegador con el programa listo para iniciar la simulación



Seleccionamos nuestro archivo xml para iniciar, para esto usamos el botón donde dice cargar archivo.



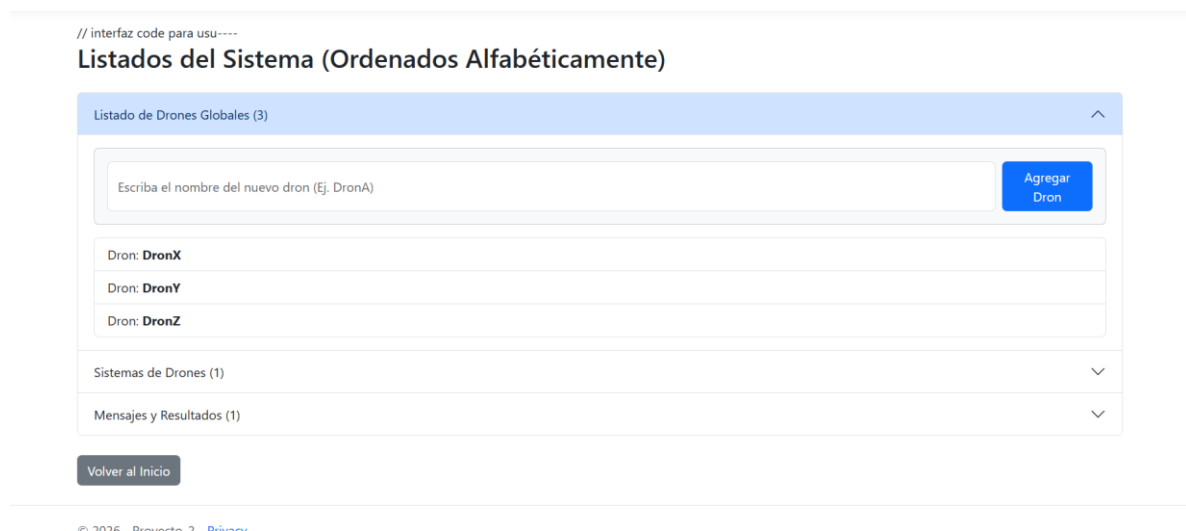
se abrirá nuestro explorador de archivos seleccionamos el archivo que deseamos simular, teniendo en cuenta que nuestro programa solo simula archivos xml por si se le introduce otro tipo de archivo este lo rechazará.



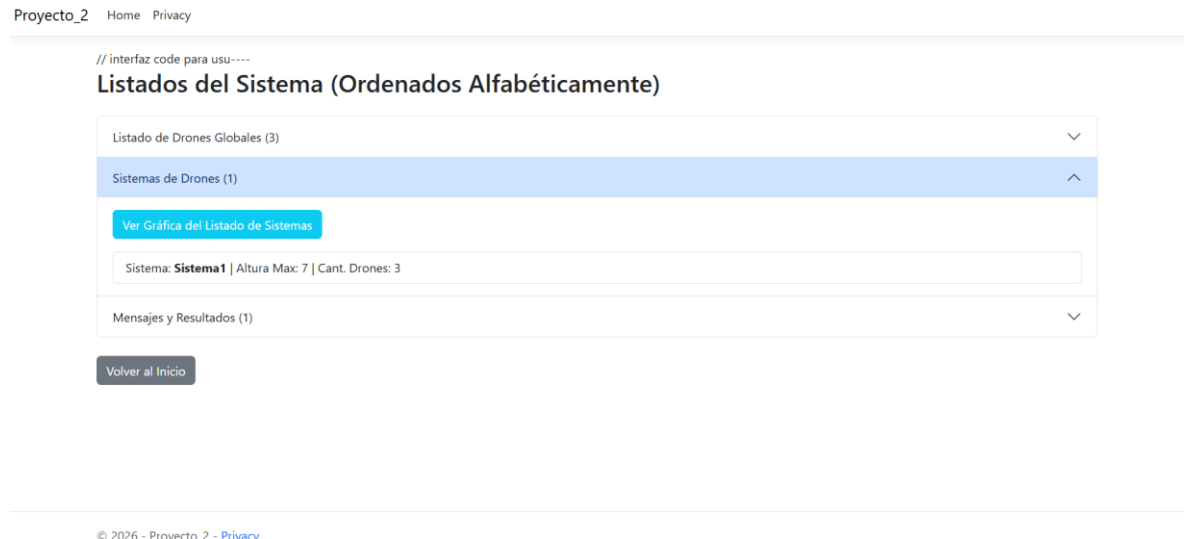
Damos en procesar simulación, nos sale una notificación indicando que la simulación se ha realizado exitosamente.
Si queremos observar la simulación ingresamos a la sección gestión esto se encuentra presionando el botón gestión en nuestra interfaz.



Vemos que se nos abre el panel de simulación, donde se encuentra informacion relacionado a los drones.



Si queremos ver toda la información de drones seleccionamos sistema de drones, que nos dará toda información de drones y el mensaje que contiene la simulación.



Para ver la grafica seleccionamos ver grafica del sistema, el programa nos ejecutara la imagen de la simulación.



Si queremos ver las instrucciones de los drones y su grafica selecciones el apartado de mensaje.

// interfaz code para usu----

Listados del Sistema (Ordenados Alfabéticamente)

Listado de Drones Globales (3) ▾

Sistemas de Drones (1) ▾

Mensajes y Resultados (1) ▴

Mensaje: **Mensaje1** | Sistema: Sistema1 | Tiempo: 7 s | Respuesta: I

Ver Detalle de Pasos

Ver Graphviz

Volver al Inicio

Vemos que podemos ver los movimientos de los drones por medio de una tabla siguiendo los pasos de los drones en la simulación.

Mensaje: **Mensaje1** | Sistema: Sistema1 | Tiempo: 7 s | Respuesta: I

Ver Detalle de Pasos

Ver Graphviz

Segundo	Dron	Acción Realizada
1	DronX	Subir
1	DronY	Subir
1	DronZ	Subir
2	DronX	Subir
2	DronY	Subir
2	DronZ	Subir
3	DronX	Emitir luz
3	DronY	Subir
3	DronZ	Esperar
4	DronX	Esperar
4	DronY	Emitir luz
4	DronZ	Esperar
5	DronX	Esperar
5	DronY	Bajar
5	DronZ	Emitir luz
6	DronX	Esperar
6	DronY	Bajar
6	DronZ	Esperar
7	DronX	Esperar
7	DronY	Emitir luz

Para ver imagen que nos genera nuestro programa seleccionamos la opción de graphviz nos genera la imagen del diagrama de nuestros drones activos.



Si queremos iniciar otra simulación regresamos a nuestro inicio de la interfaz y seleccionamos el apartado de reiniciar sistema.

// code para paginaa principiisll -----

Sistema de Drones ING

Cargar Archivo XML

Seleccione el archivo de entrada:

Sin archivos seleccionados

Acciones del Sistema

Nos aparecerá una notificación que el sistema sea reiniciado y que esta listo para otra simulación.



En el apartado de ayuda encontramos la información del desarrollador y el link del repositorio donde se encuentra los archivos del programa por si desean mejorar o arreglar errores que se puedan encontrar en las simulaciones futuras.



Conclusiones

- **Eficiencia en el Manejo de Memoria Dinámica:** La implementación de estructuras de datos lineales propias, como la ListaEnlazada<T> y la Cola<T>, demostró ser una alternativa superior a las colecciones estáticas para este tipo de problemas. Al gestionar manualmente los punteros y la instanciación de objetos de tipo Nodo, se logró un control total sobre el ciclo de vida de la información, garantizando que el sistema sea capaz de procesar configuraciones de drones de gran escala sin el overhead que suponen las librerías genéricas predefinidas. Esta práctica no solo cumple con las restricciones del curso, sino que refuerza la comprensión de la gestión de memoria a bajo nivel en C#.
- **Optimización Mediante Lógica Predictiva:** El desarrollo del algoritmo de simulación permitió concluir que la eficiencia de un enjambre de drones no depende únicamente de la velocidad de sus componentes, sino de la orquestación inteligente de sus movimientos. Al integrar una lógica de "lectura anticipada" de instrucciones, se redujo significativamente el tiempo óptimo de misión, permitiendo que los drones se posicionen en las alturas requeridas de forma simultánea al proceso de emisión de luz. Esto valida que la programación orientada a objetos es una herramienta poderosa para modelar comportamientos complejos y optimizar recursos físicos mediante software.
- **Importancia de la Interoperabilidad y Visualización:** La integración exitosa de estándares XML y la herramienta Graphviz subraya la importancia de la interoperabilidad en los sistemas de ingeniería modernos. El uso de XML permite que el sistema de drones sea alimentado por fuentes de datos externas de manera estructurada, mientras que la generación de grafos dinámicos proporciona una capa de abstracción visual indispensable para la depuración y auditoría de la lógica algorítmica. En conclusión, un software de ingeniería es verdaderamente funcional cuando, además de procesar datos, es capaz de comunicar sus resultados de forma clara y estandarizada.